

# Silent Vision

Personalized fixed-phrase recognition from a short camera clip with audio disabled.

## PROJECT PROFILE

# One silent clip. One inspectable phrase decision.

Silent Vision recognizes a small catalog of personalized fixed phrases. It is not open-vocabulary lipreading. Exact accepted text and intent come from the registered catalog.

### zh\_light\_on\_hello

你好，请帮我打开灯 -> LIGHT\_ON

### zh\_chat\_meal

你吃饭了吗？ -> CHAT\_OTHER

## CURRENT STATUS

CPU code decodes the video, detects and aligns one face, and crops the mouth. The fixed-phrase Torch model is designed to train and run on AMD Radeon through ROCm. The repository returns decisions; it does not operate an external tool or device.

## PENDING

Official Radeon training, untouched final evaluation, and recorded demonstration.

# Target users

A deliberate visual input for one user and a small registered phrase catalog.

## Creator control input

Another application can map an accepted intent to a capture, cue, or editing action.

## Accessible service

A small visual phrase set can supplement other communication channels at a service desk.

## Noisy workspace

A deliberate visual command can help when microphone recognition is unreliable.

## Privacy-sensitive room

The browser captures video without enabling the microphone or continuous audio recording.

### PRODUCT BOUNDARY

Silent Vision answers "which registered phrase best fits this visible mouth sequence?" It does not claim a transcript or cross-speaker generalization. The current source implements recognition, not a downstream creator or device action.

An integrator still defines allowed intents, confirmation rules, retention policy, and failure handling.

# Product workflow and safety

One short recording, one catalog decision, and a fail-closed rejection boundary.

**01**

## Capture

Record one 2-5 second WebM clip with audio disabled.



**02**

## Preprocess

Decode, detect one face, align, and crop the mouth on CPU.



**03**

## Classify

Run phrase logits and a normalized embedding on Radeon.



**04**

## Gate

Require checkpoint probability and phrase-centroid distance.



**05**

## Route

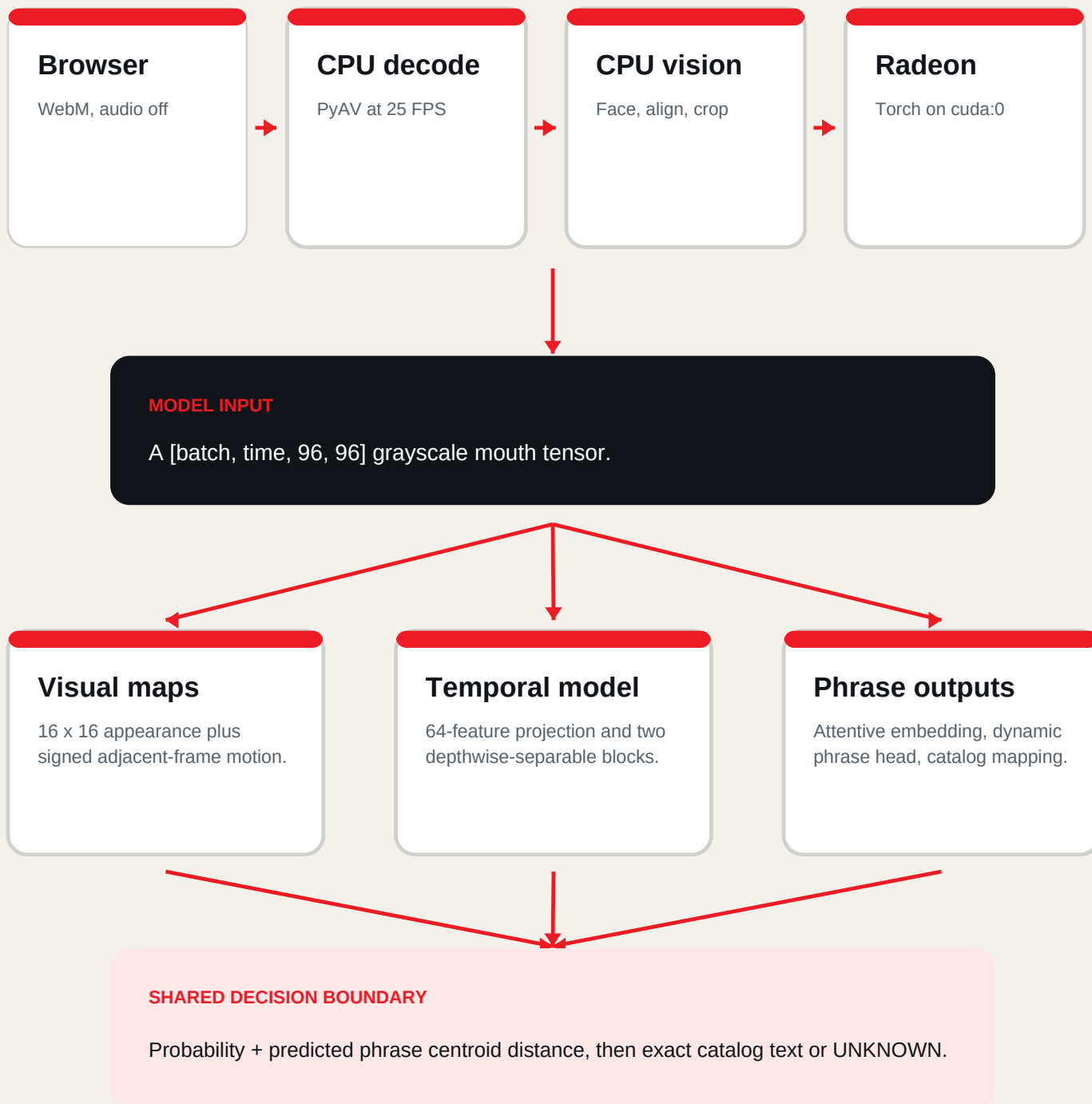
Return exact catalog text or reject the clip as UNKNOWN.

### SAFETY RULE

Both calibrated gates must pass. Rejected output carries no matched phrase text and cannot execute. Top-1 margin is diagnostic only.

# System architecture

CPU media preprocessing feeds one fixed-phrase model on AMD Radeon through ROCm.



# Model and algorithm

A small temporal model learns phrase classes and a calibrated embedding boundary.

1

## Appearance + motion

Downsample to 16 x 16 and compute signed adjacent-frame differences



2

## 64-feature projection

Learn a compact visual representation for every frame



3

## 2 temporal blocks

Residual depthwise and pointwise one-dimensional convolutions



4

## Embedding + phrase head

Attentive pooling, normalized embedding, dynamic catalog classes

### Acceptance

The top phrase must pass the global minimum probability and its maximum cosine distance from the training centroid. Margin is diagnostic only.

### Catalog boundary

UNKNOWN is not trained. Accepted exact text and mapped intent come from the checkpoint catalog; rejected output carries neither.

# AMD Radeon and ROCm

The official path fails closed unless the fixed-phrase Torch model can use ROCm cuda:0.

## CPU preprocessing

PyAV decode, 25 FPS resampling, MediaPipe face detection, alignment, and 96 x 96 mouth cropping.

## Radeon classification

PyTorch runs visual maps, temporal blocks, attentive embedding, phrase logits, and centroid comparison through ROCm.

### REPRODUCTION GUARDS

- `torch.version.hip` must be non-empty.
- `cuda:0` must be visible and accept an allocation.
- A non-empty fixed-phrase checkpoint must be provided.

### EVIDENCE STATUS

Pending: the official Radeon training run, final evaluation report, and recorded demonstration. Small-data smoke artifacts are non-evidentiary, and no accuracy or latency figure is published.

### NEXT STEPS

1. Meet the official sample gate. 2. Build hashed manifests. 3. Train and calibrate on Radeon. 4. Evaluate untouched final partitions. 5. Record the demo.

<https://github.com/fangjixin/silent-vision>